



CQRS Pattern in .NET

Short & Crispy Notes

By Logic Lense

✓ Interview Ready

⚡ Quick Revision

🔗 .NET Focused

12 Sections • Code Examples • Cheat Sheet

Table of Contents

- 1** What is CQRS?
- 2** Core Concept — Command vs Query
- 3** Why Use CQRS in .NET?
- 4** Basic Architecture Overview
- 5** Command Side (Write Operations)
- 6** Query Side (Read Operations)
- 7** CQRS with MediatR in ASP.NET Core
- 8** CQRS Folder Structure (.NET Project)
- 9** CQRS vs Traditional CRUD
- 10** Benefits & Drawbacks
- 1 1** Real-world Use Cases
- 1 2** Interview Quick Revision
-  Quick Cheat Sheet

1 What is CQRS?

Definition (Simple Language)

CQRS stands for Command Query Responsibility Segregation.

It is a pattern that SEPARATES the operations that READ data from the operations that WRITE (change) data.

Instead of one model/service handling everything, you have TWO separate models:

- Command Model → handles all changes (Create, Update, Delete)
- Query Model → handles all reads (Select / Get data)

? Why CQRS Exists

In traditional apps, one model/service does everything — read and write. As apps grow:

- Reads and writes have VERY different performance needs
- Write operations need validation, business rules, and transactions
- Read operations need speed, joins, flat data — no heavy logic
- Mixing both in one place creates complex, hard-to-maintain code

	Without CQRS	With CQRS
1	✗ One model for read + write	✓ Separate read & write models
2	✗ Complex, bloated services	✓ Clean, focused handlers
3	✗ Hard to scale independently	✓ Scale reads & writes separately
4	✗ Mixed responsibilities	✓ Single Responsibility per handler

2 Core Concept — Command vs Query

● COMMAND

Changes Data

- Creates new records
- Updates existing data
- Deletes records
- Returns void or minimal result
- Has business validation
- May trigger domain events

Examples:

- CreateOrderCommand
- UpdateUserCommand
- DeleteProductCommand

● QUERY

Reads Data

- Fetches records
- Returns data/DTOs
- Never modifies state
- Can be cached easily
- Optimized for speed
- No side effects

Examples:

- GetOrderByIdQuery
- GetAllUsersQuery
- SearchProductsQuery

⚡ The Golden Rule of CQRS

"A function should either CHANGE the state of the system, OR return data — but NEVER BOTH."

— Bertrand Meyer (CQS Principle, which CQRS is based on)

📦 Separation of Responsibilities

Each handler has ONE job. It either reads OR writes. This eliminates the confusion of side-effects.

- ▶ Commands: focused on intent + validation + persistence
- ▶ Queries: focused on projection + performance + data shaping
- ▶ No shared model between the two sides — they evolve independently

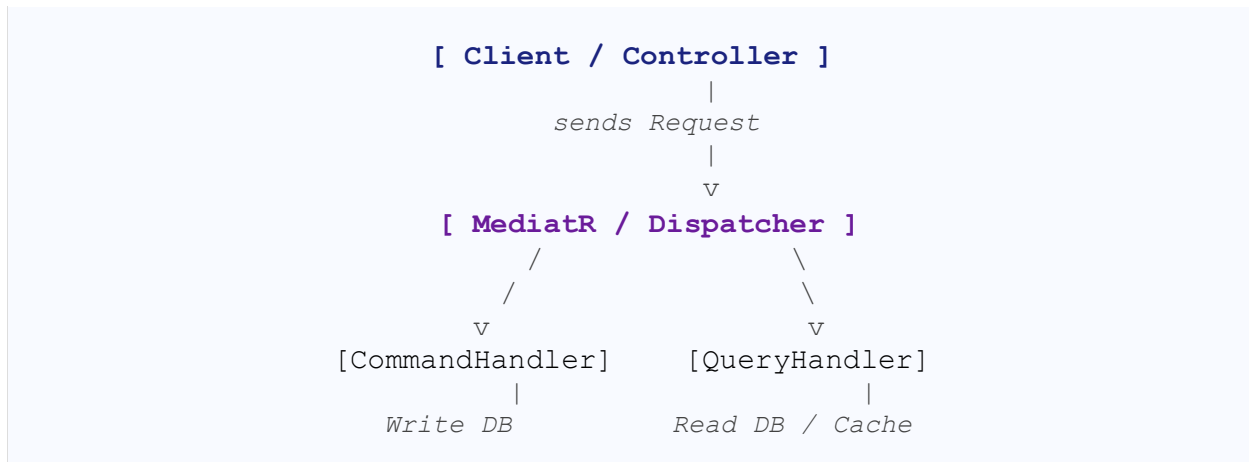
3

Why Use CQRS in .NET?

	Benefit	What It Means
	Clean Code Structure	Each handler has one responsibility. Easy to read, test, and extend.
	Better Scalability	Read and write services can scale independently based on load.
	Easy Maintenance	Changing a query won't break a command. Isolated changes.
	Performance Optimization	Read models can use raw SQL / Dapper / cached projections — no ORM overhead.
	Testability	Unit test each handler in isolation without touching the entire service.
	Security Clarity	Easy to apply different auth/validation rules for reads vs writes.

4 Basic Architecture Overview

Request Flow Diagram



Command Handler

Receives the Command object
Validates input data
Applies business logic
Saves to Write Database
Returns success/failure
May publish domain events



Query Handler

Receives the Query object
Fetches from Read Database
Maps to DTO / ViewModel
Returns data to caller
No business rules applied
Optimized for performance

Data Layer Interaction

CQRS allows using DIFFERENT databases or data access strategies for reads and writes:

- ▶ Write side: Entity Framework Core (EF Core) with full ORM tracking
- ▶ Read side: Dapper with raw SQL for fast, flat projections
- ▶ Advanced: Separate read replica / read-optimized store (Event Sourcing)

5 Command Side (Write Operations)

What is a Command?

A Command is an object that REPRESENTS an INTENTION to change the system state. It carries the data needed to perform the write operation.

Naming convention: Verb + Entity + 'Command' → CreateOrderCommand, DeleteUserCommand

Command Model Example

```
// Command - carries the intent + data
public class CreateProductCommand : IRequest<int>
{
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int StockQuantity { get; set; }
    public string Category { get; set; }
}
```

Command Handler Example

```
// Handler - processes the command
public class CreateProductCommandHandler
    : IRequestHandler<CreateProductCommand, int>
{
    private readonly IAppDbContext _context;

    public CreateProductCommandHandler(IAppDbContext context)
        => _context = context;

    public async Task<int> Handle(
        CreateProductCommand request,
        CancellationToken cancellation_token)
    {
        var product = new Product
        {
            Name = request.Name,
            Price = request.Price,
            Stock = request.StockQuantity
        };

        _context.Products.Add(product);
        await _context.SaveChangesAsync(cancellation_token);

        return product.Id; // returns new ID
    }
}
```

✓ Validation Idea

Use FluentValidation with a Pipeline Behavior to validate BEFORE the handler runs:

```
public class CreateProductValidator
    : AbstractValidator<CreateProductCommand>
{
    public CreateProductValidator()
    {
        RuleFor(x => x.Name).NotEmpty().MaximumLength(100);
        RuleFor(x => x.Price).GreaterThan(0);
        RuleFor(x => x.StockQuantity).GreaterThanOrEqualTo(0);
    }
}
```


6 Query Side (Read Operations)

What is a Query?

A Query is an object that REPRESENTS a REQUEST for data. It does NOT change state. Queries return DTOs (Data Transfer Objects) — not domain entities.
Naming convention: 'Get' + Entity + 'Query' → GetProductByIdQuery, GetAllOrdersQuery

Query Model Example

```
// Query - carries the search criteria
public class GetProductByIdQuery : IRequest<ProductDto>
{
    public int ProductId { get; set; }

    public GetProductByIdQuery(int id)
        => ProductId = id;
}

// DTO - optimized read model (flat, no domain logic)
public class ProductDto
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public string CategoryName { get; set; } // pre-joined!
    public bool IsInStock { get; set; }
}
```

Query Handler Example

```
public class GetProductByIdQueryHandler
    : IRequestHandler<GetProductByIdQuery, ProductDto>
{
    private readonly IDbConnection _connection; // Dapper

    public GetProductByIdQueryHandler(IDbConnection connection)
        => _connection = connection;

    public async Task<ProductDto> Handle(
        GetProductByIdQuery request,
        CancellationToken cancellationToken)
    {
        var sql = @"
            SELECT p.Id, p.Name, p.Price,
                   c.Name AS CategoryName,
                   CASE WHEN p.Stock > 0 THEN 1 ELSE 0 END AS IsInStock
            FROM Products p
            JOIN Categories c ON p.CategoryId = c.Id
            WHERE p.Id = @ProductId";

        return await _connection.QueryFirstOrDefaultAsync<ProductDto>(
```

```
        sql, new { request.ProductId });  
    }  
}
```

Logic Lense

? Why MediatR is Used

- MediatR is a lightweight .NET library that implements the Mediator pattern
- It acts as an in-process message bus — dispatches Commands and Queries to their handlers
- Decouples your controllers from your business logic handlers
- Enables Pipeline Behaviors for cross-cutting concerns (logging, validation, caching)

📦 Installation

```
dotnet add package MediatR
dotnet add package MediatR.Extensions.Microsoft.DependencyInjection
```

⚙️ Registration in Program.cs

```
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(Program).Assembly));
```

🔧 IRequest & Handler Basics

```
// IRequest<TResponse> – marks a class as a Command or Query
// IRequestHandler<TRequest, TResponse> – processes it

// Command (returns int – the new ID)
public class CreateUserCommand : IRequest<int> { ... }

// Query (returns UserDto)
public class GetUserQuery : IRequest<UserDto> { ... }

// Controller usage – clean and simple
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly IMediator _mediator;
    public UsersController(IMediator mediator) => _mediator = mediator;

    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreateUserCommand cmd)
        => Ok(await _mediator.Send(cmd));

    [HttpGet("{id}")]
    public async Task<IActionResult> Get(int id)
        => Ok(await _mediator.Send(new GetUserQuery(id)));
}
```

📄 Simple Flow

Controller → `_mediator.Send(command/query)`

- MediatR finds the correct Handler
- `Handler.Handle()` executes
- Result returned to Controller
- Controller returns HTTP Response

Logic Lense

8 CQRS Folder Structure (.NET Project)

```
src/
├── MyApp.API/
│   └── Controllers/
│       ├── OrdersController.cs
│       └── ProductsController.cs
├── MyApp.Application/
│   ├── Commands/
│   │   ├── CreateOrder/
│   │   │   ├── CreateOrderCommand.cs      ← Command model
│   │   │   ├── CreateOrderCommandHandler.cs ← Handler
│   │   │   └── CreateOrderValidator.cs     ← FluentValidation
│   │   ├── UpdateOrder/
│   │   │   ├── UpdateOrderCommand.cs
│   │   │   └── UpdateOrderCommandHandler.cs
│   │   └── DeleteOrder/
│   │       ├── DeleteOrderCommand.cs
│   │       └── DeleteOrderCommandHandler.cs
│   ├── Queries/
│   │   ├── GetOrderById/
│   │   │   ├── GetOrderByIdQuery.cs      ← Query model
│   │   │   └── GetOrderByIdQueryHandler.cs ← Handler
│   │   └── GetAllOrders/
│   │       ├── GetAllOrdersQuery.cs
│   │       └── GetAllOrdersQueryHandler.cs
│   ├── DTOs/
│   │   ├── OrderDto.cs
│   │   └── OrderSummaryDto.cs
│   ├── Behaviors/                                ← Pipeline Behaviors
│   │   ├── ValidationBehavior.cs
│   │   └── LoggingBehavior.cs
├── MyApp.Domain/
│   ├── Entities/
│   └── Interfaces/
└── MyApp.Infrastructure/
    ├── Persistence/                ← EF Core (Write)
    └── ReadModels/                 ← Dapper / Read
```

9 CQRS vs Traditional CRUD

Aspect	Traditional CRUD	CQRS
Data Model	One model for read & write	Separate models for each
Service Layer	Single service with all methods	Separate Command/Query handlers
Scalability	Scales as one unit	Reads & writes scale independently
Complexity	Simple, easy to start	Higher initial setup
Performance	Shared ORM for everything	Optimized per operation type
Testing	Test entire service class	Test each handler in isolation
Maintenance	Changes affect all operations	Isolated changes per handler
Best For	Small/simple CRUD apps	Complex, high-traffic systems

✓ When CQRS is Better

- Large enterprise applications
- High read vs write ratio
- Different teams own read/write
- Complex business domain
- Microservices architecture
- Event Sourcing needed

✓ When CRUD is Enough

- Small / simple apps
- CRUD with basic data
- Small team, fast delivery
- Low traffic, low complexity
- Admin panels / dashboards
- MVP / Prototype projects

10 Benefits & Drawbacks

✓ Pros

	Scalability	Read and write sides scale independently. High-read apps can add read replicas without touching write logic.
	Single Responsibility	Each handler has one job. Follows SOLID principles naturally.
	Performance	Read side can bypass ORM, use caching, flat projections — much faster queries.
	Testability	Each handler is a small, focused class — easy to unit test with no side effects.
	Flexibility	Change read model without touching domain logic. Add new queries freely.
	Security	Different auth policies for read vs write operations are straightforward.

✗ Cons

	Complexity	More classes, more files, more layers. Steeper learning curve for new developers.
	Code Overhead	For simple CRUD, CQRS adds unnecessary structure with little benefit.
	Consistency	If using separate read/write DBs, achieving consistency requires eventual consistency patterns.
	Setup Time	Initial setup (MediatR, handlers, DTOs, behaviors) takes more time than simple services.
	Overkill Risk	For small or simple apps, CQRS is over-engineering and reduces developer velocity.

1

1

Real-world Use Cases

 Large Enterprise	 High-Read Systems	 Microservices
E-Commerce Platforms → Complex order processing → Inventory management → Multi-team development Banking/Finance Systems → Transaction commands → Statement queries → Audit trail support ERP/CRM Systems → Multiple modules → Complex workflows	Product Catalogs → 1000x more reads than writes → Cached read projections Social Media Feeds → Feed queries optimized → Post commands separate Reporting Systems → Complex read aggregates → No write interference Search Engines → Read side uses Elastic → Write side uses SQL DB	Event-Driven Systems → Commands emit events → Services react to events → Decoupled architecture API Gateway Pattern → Different read endpoints per microservice Event Sourcing + CQRS → Commands → Events stored → Queries → Projections → Full audit history

1 2 Interview Quick Revision

One-line Definition

"CQRS separates the reading and writing of data into two distinct models — Commands for writes, Queries for reads — to improve scalability, performance, and code clarity."

Common Interview Questions

Question	Quick Answer
What is CQRS?	Pattern that separates read (Query) and write (Command) operations into different models/handlers.
Why use CQRS?	For scalability, cleaner code, independent optimization of reads and writes.
What is MediatR?	An in-process mediator library that dispatches Commands/Queries to their handlers.
Difference between CQS and CQRS?	CQS is method-level separation. CQRS is system/architecture level separation.
Can CQRS use one database?	Yes! You don't need separate DBs. CQRS is about model separation, not physical DB separation.
What's a Command Handler?	A class implementing IRequestHandler that processes a write operation.
What's a Pipeline Behavior?	MediatR middleware for cross-cutting concerns (logging, validation) applied before/after handlers.
When NOT to use CQRS?	Small simple apps, CRUD apps with no complex domain logic, MVP/prototype phases.

Common Mistakes to Avoid

	✗ Mistake	✓ Correct Approach
1	Returning data AND modifying state in a Command	Return only void, ID, or success flag from Commands
2	Using domain entities as Query return types	Return DTOs — flat, optimized read models
3	Making Queries trigger side effects	Queries must be pure reads with no state changes
4	Using CQRS everywhere by default	Evaluate complexity — CRUD is fine for simple apps

5

Forgetting validation in Commands

Always validate with FluentValidation + Pipeline Behavior

Logic Lense

☆ Quick Cheat Sheet — 1 Page Summary

🌀 COMMAND = WRITE

- Changes system state
- Returns void / ID only
- Has business validation
- Uses Write DB (EF Core)
- Example: *CreateOrderCommand*

🌀 QUERY = READ

- Never modifies state
- Returns DTOs / data
- Optimized for speed
- Uses Read DB (Dapper)
- Example: *GetOrderByIdQuery*

🚫 NEVER MIX BOTH 🚫

A function either CHANGES state OR RETURNS data — NEVER BOTH

📐 Single Responsibility Flow

Controller

```
_mediator.Send(new CreateXCommand(...)) → CommandHandler → Write DB  
_mediator.Send(new GetXQuery(...)) → QueryHandler → Read DB
```



MediatR

IRequest<T>
IRequestHandler<>
_mediator.Send()



Validation

FluentValidation
Pipeline Behavior
Runs before Handler



Structure

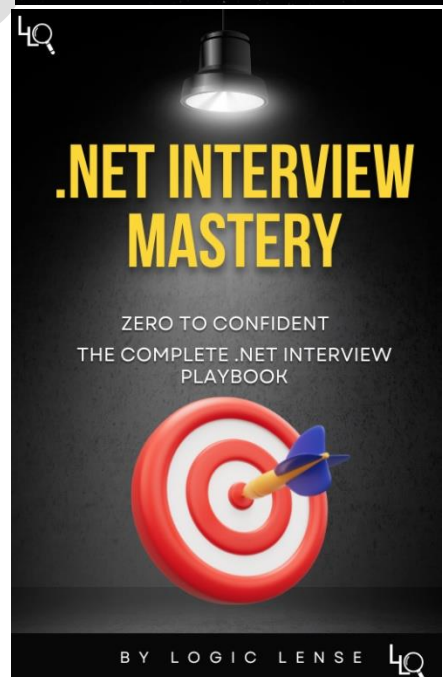
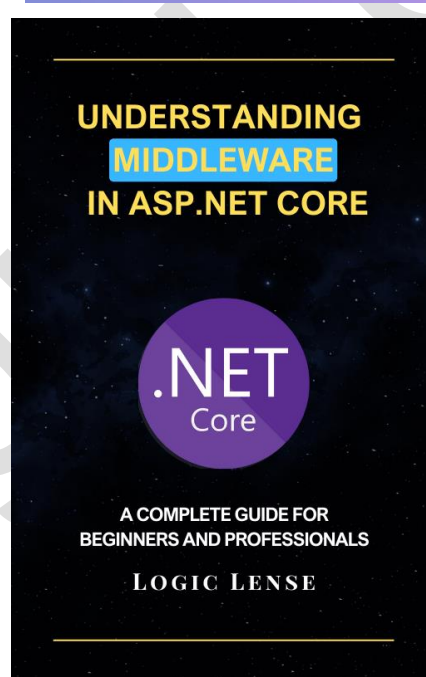
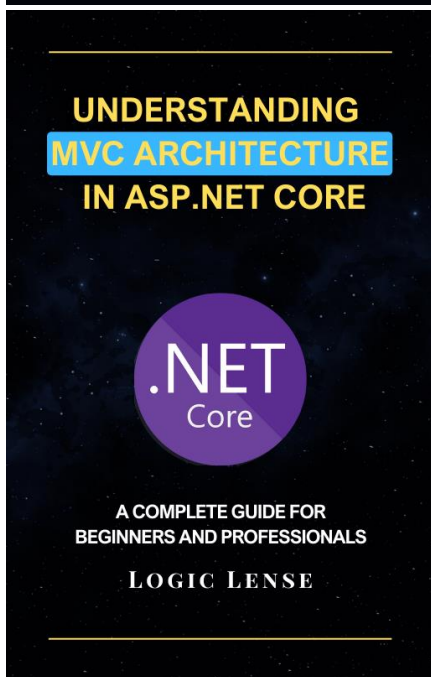
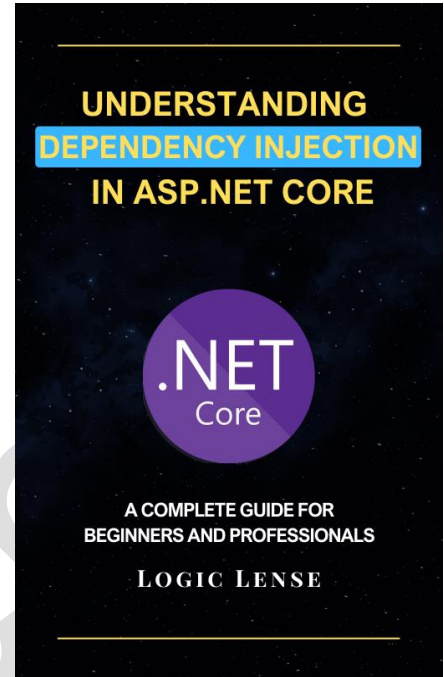
Commands/ folder
Queries/ folder
DTOs/ folder

☆ Remember This Always

CQRS is not about databases — it's about RESPONSIBILITY SEPARATION

Command = Intent to CHANGE | Query = Intent to READ

★ RECOMMENDED EBOOKS YOU WILL LOVE



We Would love to have you as a learner 💖! 🌟

★ STAY CONNECTED & LEARN DAILY

Follow Logic Lense for daily content, tips, jobs, projects, and coding knowledge:

Share your feedback at: - <mailto:support@logiclense.com>



★ A FINAL NOTE

The tech world rewards **consistent learners**.
Stay committed. Stay curious. Keep building.
Your dream job is closer than you think.

See you in the next product.

Keep growing with Logic Lense. ❤️